



The Design and Analysis of Algorithms



Course Prerequisite

- Data Structure
- Discrete Mathematics
- Knowledge of any programming language

Syllabus :

UNIT 1:	
Introduction : Notion of Algorithm, Fundamentals of Algorithmic problem solving, Important problem types, Fundamental data structures. Fundamentals of the Analysis of Algorithm Efficiency : Analysis Framework, Asymptotic Notations and Basic efficiency classes, Mathematical Analysis of Recursive and Non-recursive algorithms, Examples	7 Hours
UNIT 2:	
Brute Force : Selection sort and Bubble sort, Sequential search and String matching, Exhaustive search Divide and Conquer : Merge sort ,Quick sort	8 Hours
UNIT 3:	
Decrease-and-Conquer : Algorithm for generating combinatorial objects, Decrease by a constant factor algorithms. Transform-and Conquer : Balanced Search Trees, Heaps and Heap sort.	8 Hours
UNIT 4:	
Space and Time Tradeoffs - Sorting by Counting, Input Enhancement in String Matching. Dynamic Programming : Warshall's__and Floyd's Algorithms, Multistage Graph.	8 Hours
UNIT 5:	
Greedy Technique : Prim's Algorithm, Dijkstra's Algorithm, Huffman Trees. Coping with the Branch and Bound : Limitations of Algorithm Power-Backtracking, Branch and Bound.	8 Hours



Reference Books:

- 1)Anany Levitin, “Introduction to The Design and Analysis of Algorithms”, 2nd edition, Pearson Education, 2011.**
- 2)Thomas H. Cormen, Charles E. Leiserson, Ronal L. Rivest and Clifford Stein, “Introduction to Algorithms”, 2nd edition, Prentice-Hall India, 2009.**
- 3)Horowitz E., Sahni S., RajasekaranS.,“Computer Algorithms”, 2nd edition, Universities Press, 2013.**



CHAPTER 1: INTRODUCTION

- **What is an Algorithm (Notion of Algorithm)**
- **Steps in Designing and Implementing an Algorithm**
(Fundamentals of Algorithmic problem solving)
- **Important Problem Types**
- **Fundamental Data Structures**



Algorithm:

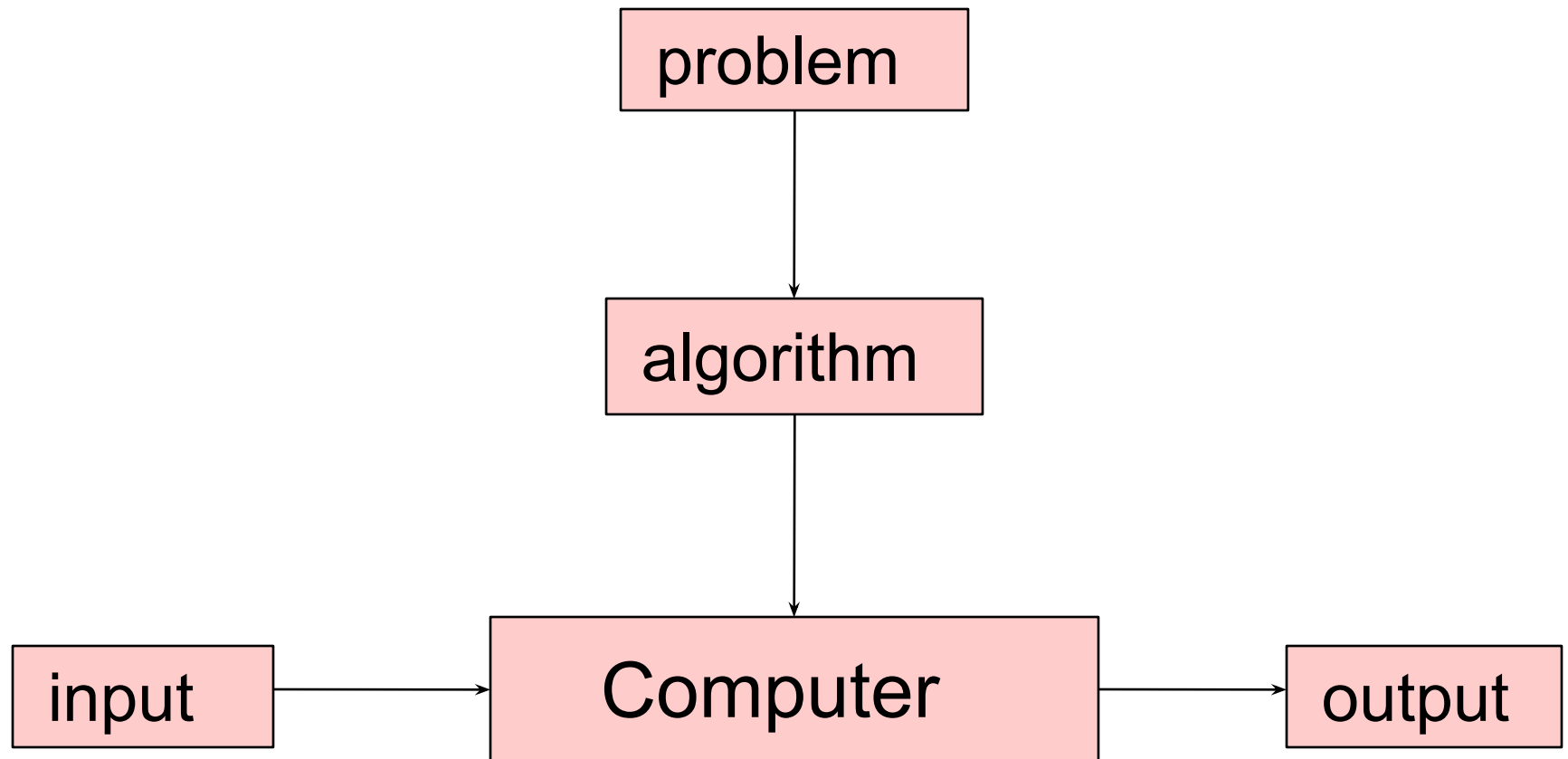
- An algorithm is a set of rules for carrying out calculation either by hand or on a machine.
- An algorithm is a finite step-by-step procedure to achieve a required result.
- An algorithm is a sequence of computational steps that transform the input into the output.
- An algorithm is a sequence of operations performed on data that have to be organized in data structures.
- An algorithm is an abstraction of a program to be executed on a physical machine (model of Computation).



ALGORITHM

- **A sequence of unambiguous instructions for solving a problem, i.e. for obtaining the required output for any legitimate input in a finite amount of time**

Notion of algorithm :



Characteristics of an Algorithm :

An algorithm is a method , technique, procedure, routine,... to solve a problem with following requirements:

- **Finiteness**

Algorithm terminates after a finite number of steps

- **Definiteness**

Each step must be carefully and unambiguously specified.

- **Input**

It consists of zero or more valid inputs , which are clearly specified.

- **Output**

An algorithm has one or more outputs. It should produce the correct output given a valid input.

- **Effectiveness**

Algorithm steps must be sufficiently simple and basic.



Important points :

- **Non-ambiguity**
- **Range of inputs**
- **The same algorithm can be represented in different ways**
- **Several algorithms for solving the same problem**

Example : Algorithm for computing $\text{gcd}(m,n)$

Euclid's algorithm :

Step 1. If $n=0$, return the value of m as the answer and stop; otherwise proceed to step 2.

Step 2. Divide m by n and assign the value of the remainder to r .

Step 3. Assign the value of n to m and the value of r to n . Go to Step 1.

Pseudocode of Euclid's Algorithm

Algorithm *Euclid*(m, n)

//Computes gcd (m, n) by Euclid's algorithm

//Input: Two nonnegative, not-both-zero integers m and n

//Output: Greatest common divisor of m and n

while $n \neq 0$ do

{

$r \leftarrow m \bmod n$

$m \leftarrow n$

$n \leftarrow r$

}

return m

■ Questions:

- Finiteness: how do we know that Euclid's algorithm actually comes to a stop?
- Definiteness: nonambiguity
- Effectiveness: effectively computable.

Consecutive integer checking steps

Step 1. Assign the value of $\min(m,n)$ to t .

Step 2. Divide m by t . If the remainder of this division is 0, go to step 3; otherwise go to Step 4.

Step 3. Divide n by t . if the remainder of this division is 0, return the value of the answer and stop; otherwise proceed to Step 4.

Step 4. Decrease the value of t by 1.
Go to Step 2.

1. $t \leftarrow \min(m,n)$

2. **if** $m \% t = 0$
go to 3,

else go to 4

3. **if** $n \% t = 0$
return t ,
else go to 4

4. $t \leftarrow t - 1$

5. **Go to** 2



Middle-school procedure

Step 1. Find the prime factors of m .

Step 2. Find the prime factors of n .

Step 3. Identify all the common factors in the prime expansions found in Step 1 and Step 2.

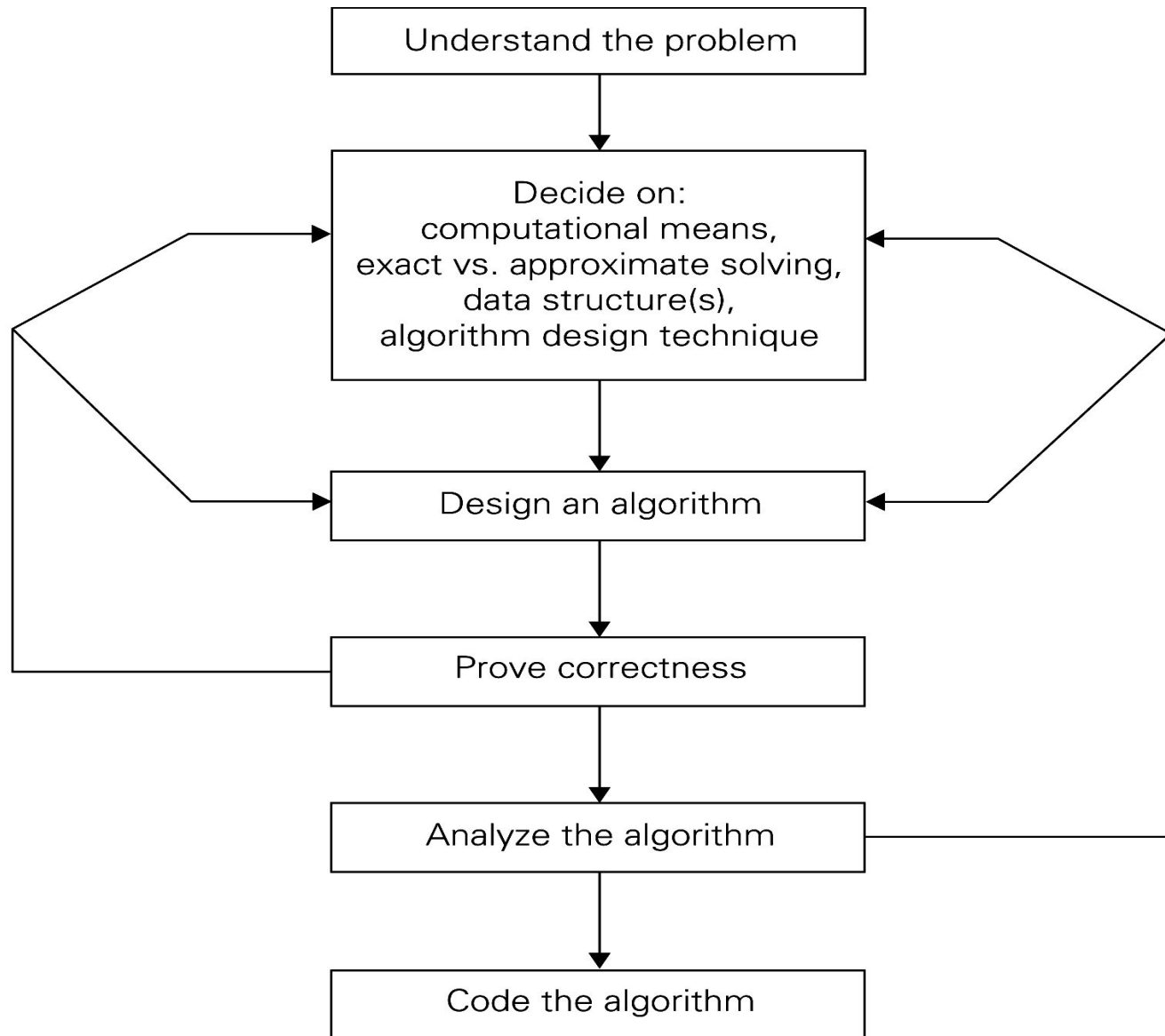
Step 4. Compute the product of all the common factors and return it as the greatest common divisor of the numbers given.



What can we learn from the previous 3 examples?

- Each step of an algorithm must be unambiguous.
- The same algorithm can be represented in several different ways. (different pseudo codes)
- There might exist more than one algorithm for a certain problem.
- Algorithms for the same problem can be based on very different ideas and can solve the problem with significantly different speeds.

Fundamentals of Algorithmic Problem solving:





What does it mean to understand the problem?

- What are the problem objects?
- What are the operations applied to the objects?
- Specify exact range of instances the algorithm needs to handle
- A correct algorithm is not one that works most of the time but one that works correctly for all legitimate inputs

Choosing between Exact and Approximate Problem solving

Approximation algorithm : Problems that cannot be solved exactly

ex. Extracting square roots, solving nonlinear equations, evaluating definite integrals

Exact algorithm : **Ex : TSP**

Deciding upon appropriate data structures

Algorithm + Data structure = Programs

Two main issues related to algorithms

■ How to design algorithms :

An algorithm design technique is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.

- Brute force
- Divide and conquer
- Decrease and conquer
- Transform and conquer
- Space and time tradeoffs
- Greedy approach
- Dynamic programming
- Backtracking
- Branch and bound

...Algorithm design

Method of specifying an Algorithm

Pseudocode : A mixture of natural language and programming language constructs

Flowchart : Diagrammatic representation of the algorithm

- How to analyze algorithm efficiency



Prove correctness

- **Correct output for every legitimate input in finite amount of time**
- **Based on correct math formula**
- **By induction**

Analyze the algorithm (Theoretical and Empirical Analysis)

Efficiency : Time and Space

Simplicity :

Generality : range of inputs, special cases

Optimality : no other algorithm can do better

Coding

How the objects and operations in the algorithm are represented in the chosen programming language?



Ascertaining the Capabilities of a Computational Devices

- How the objects would be represented?
- How the operations would be implemented?
 - Sequential algorithms
 - Parallel algorithms

Example of computational problem: sorting

■ **Statement of problem:**

□ *Input:* A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$

□ *Output:* A reordering of the input sequence $\langle a'_1, a'_2, \dots, a'_n \rangle$ so that $a'_i \leq a'_j$ whenever $i < j$

■ **Instance:** The sequence $\langle 5, 3, 2, 8, 3 \rangle$

■ **Algorithms:**

□ Selection sort

□ Merge sort

□ Insertion sort

□ (many others)

Selection Sort

- Input: array $a[1], \dots, a[n]$
- Output: array a sorted in non-decreasing order
- Algorithm: for $i=1$ to n
swap $a[i]$ with smallest of $a[i], \dots, a[n]$



Important problem types

- Sorting
- Searching
- String processing
- Graph problems
- Combinatorial problems
- Geometric problems
- Numerical problems

Sorting

- Rearrange the items of a given list in ascending/descending order.
 - Input: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$
 - Output: A reordering $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$.
- Why sorting?
 - Help searching
 - Algorithms often use sorting as a key subroutine.
- Sorting key
 - A specially chosen piece of information used to guide sorting. I.e., sort student records by names.



Sorting

- Examples of sorting algorithms
 - Bubble sort
 - Selection sort
 - Insertion sort
 - Merge sort
 - Quick sort
 - Heap sort ...
- Evaluate sorting algorithm complexity: the number of key comparisons.
- Two properties
 - **Stability** : A sorting algorithm is called **stable** if it preserves the relative order of any two equal elements in its input.
 - **In place** : A sorting algorithm is **in place** if it does not require extra memory, except, possibly for a few memory units.

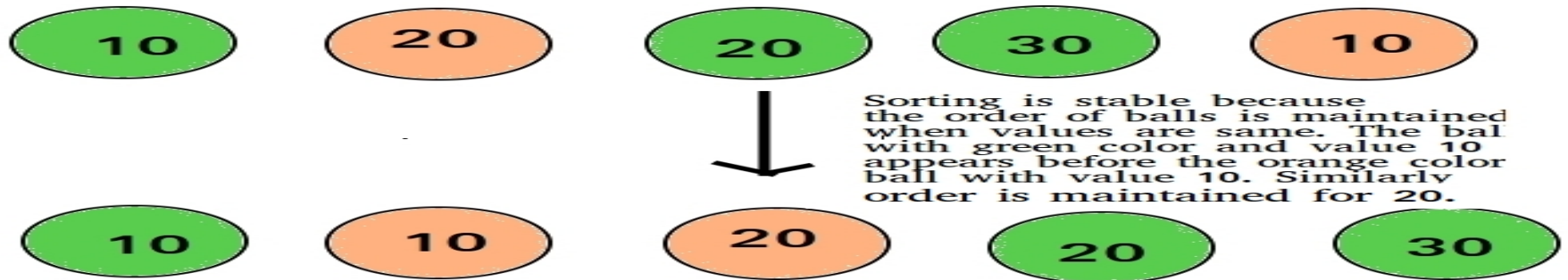
Two Important properties of sorting algorithms:

Stability : A sorting algorithm is called **stable** if it preserves the relative order of any two equal elements in its input.

(A sorting algorithm is said to be stable if two objects with equal keys appear in the same order in sorted output as they appear in the input array to be sorted)

In place : A sorting algorithm is **in place** if it does not require extra memory, except, possibly for a few memory units.

Example:



Stable Vs Unstable Sorts





Searching

- Find a given value, called a **search key**, in a given set.
- Examples of searching algorithms
 - Sequential searching
 - Binary searching...

String Processing

- A string is a sequence of characters from an alphabet.
- Text strings: letters, numbers, and special characters.
- String matching: searching for a given word/pattern in a text.



Graph Problems

- Informal definition

- A graph is a collection of points called **vertices**, some of which are connected by line segments called **edges**.

- Modeling real-life problems

- Modeling WWW
- communication networks
- Project scheduling ...

- Examples of graph algorithms

- Graph traversal algorithms
- Shortest-path algorithms
- Topological sorting



Fundamental Data Structures

- Fundamental data structure
 - Linear data structures
 - Stacks, queues, and heaps
 - Graphs
 - Trees

Linear Data Structures

■ Arrays :

- A sequence of 'n' items of the same data type that are stored contiguously in computer memory and made accessible by specifying a value of the array's index.

■ Linked List :

- A sequence of zero or more nodes each containing two kinds of information: some data and one or more links called pointers to other nodes of the linked list.
- Singly linked list (next pointer)
- Doubly linked list (next + previous pointers)

■ Arrays

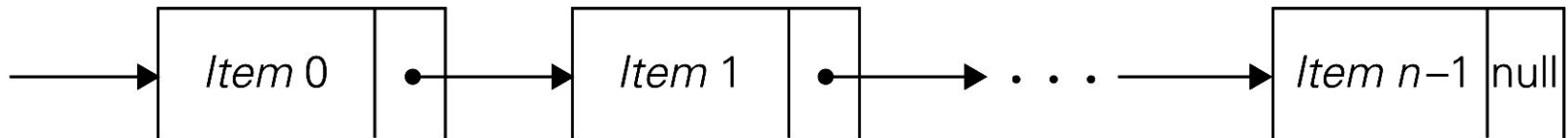
- | | | |
|-----------------|---|-----------------------------|
| ■ fixed length | , | contiguous memory locations |
| ■ direct access | , | Insert/delete |

■ Linked Lists

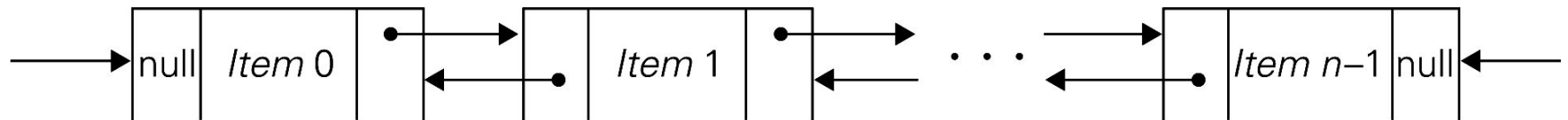
- | | | |
|-----------------------------|---|----------------------------|
| ■ dynamic length | , | arbitrary memory locations |
| ■ access by following links | , | Insert/delete |



Array of n elements



Singly linked list of n elements



Double linked list of n elements



Stacks, Queues

■ Stacks

- A stack of plates
 - insertion/deletion can be done only at the top.
 - LIFO/FILO
- Two operations (push and pop)

■ Queues

- Two operations (enqueue and dequeue)
- A queue of customers waiting for services
 - Insertion/enqueue from the rear and deletion/dequeue from the front.
 - FIFO



Priority queue:

- Priority queues (implemented using heaps)
 - A data structure for maintaining a set of elements, each associated with a key/priority, with the following operations
 - Finding the element with the highest priority
 - Deleting the element with the highest priority
 - Inserting a new element
 - Scheduling jobs on a shared computer.

Graphs

- Formal definition

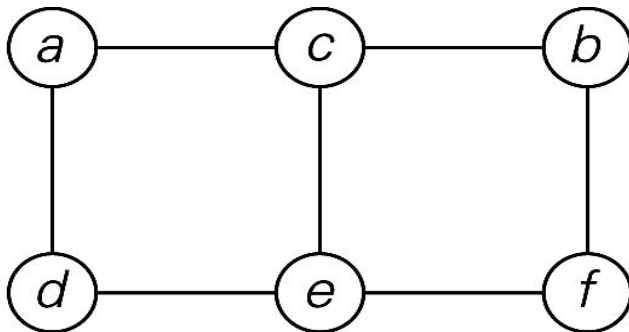
- A graph $G = \langle V, E \rangle$ is defined by a pair of two sets: a finite set V of items called **vertices** and a set E of vertex pairs called **edges**.

- **Undirected** and **directed** graphs (**digraph**).

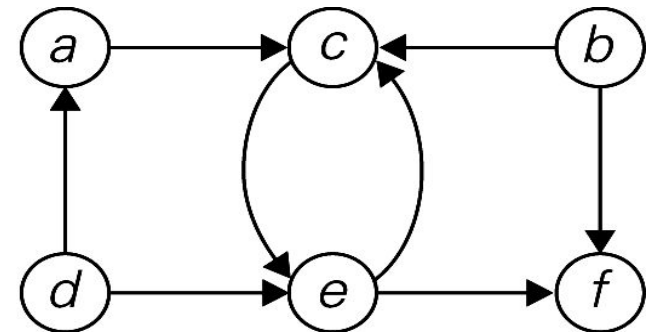
- What's the maximum number of edges in an undirected graph with $|V|$ vertices?

- **Complete**, **sparse** graph

- A graph with every pair of its vertices connected by an edge is called complete. K_n



(a)



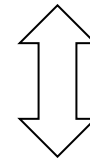
(b)

Graph Representation

■ Adjacency matrix

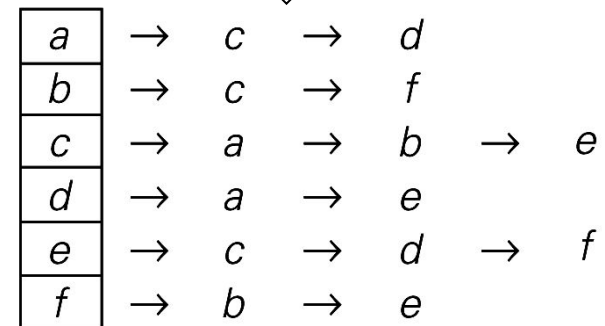
- $n \times n$ boolean matrix if $|V|$ is n .
- The element on the i^{th} row and j^{th} column is 1 if there's an edge from i^{th} vertex to the j^{th} vertex; otherwise 0.
- The adjacency matrix of an undirected graph is symmetric.

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
<i>a</i>	0	0	1	1	0	0
<i>b</i>	0	0	1	0	0	1
<i>c</i>	1	1	0	0	1	0
<i>d</i>	1	0	0	0	1	0
<i>e</i>	0	0	1	1	0	1
<i>f</i>	0	1	0	0	1	0



■ Adjacency linked lists

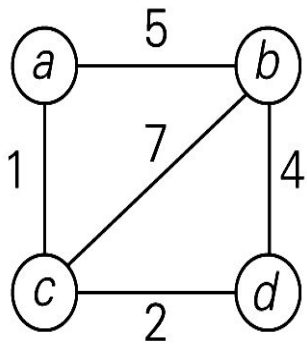
- A collection of linked lists, one for each vertex, that contain all the vertices adjacent to the list's vertex.



Weighted Graphs

■ Weighted graphs

- Graphs or digraphs with numbers assigned to the edges.



(a)

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
<i>a</i>	∞	5	1	∞
<i>b</i>	5	∞	7	4
<i>c</i>	1	7	∞	2
<i>d</i>	∞	4	2	∞

(b)

<i>a</i>	$\rightarrow b, 5 \rightarrow c, 1$
<i>b</i>	$\rightarrow a, 5 \rightarrow c, 7 \rightarrow d, 4$
<i>c</i>	$\rightarrow a, 1 \rightarrow b, 7 \rightarrow d, 2$
<i>d</i>	$\rightarrow b, 4 \rightarrow c, 2$

(c)

(a) Weighted graph. (b) Its weight matrix. (c) Its adjacency lists.

Graph Properties -- Paths and Connectivity

■ Paths

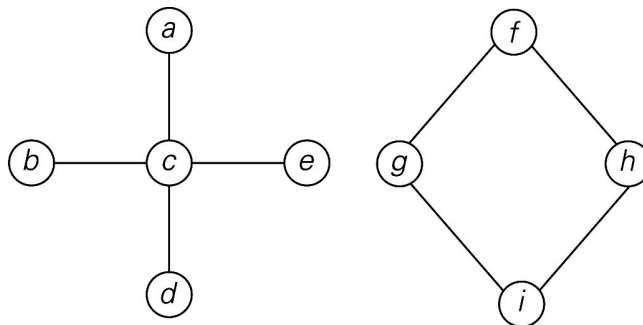
- A path from vertex u to v of a graph G is defined as a sequence of adjacent (connected by an edge) vertices that starts with u and ends with v .
- **Simple paths:** All edges of a path are distinct.
- Path lengths: the number of edges, or the number of vertices $- 1$.

■ Connected graphs

- A graph is said to be connected if for every pair of its vertices u and v there is a path from u to v .

■ Connected component

- The maximum connected subgraph of a given graph.



Graph that is not connected



Graph Properties -- Acyclicity

■ Cycle

- A simple path of a positive length that starts and ends at the same vertex.

■ Acyclic graph

- A graph without cycles
- DAG (Directed Acyclic Graph)

Trees

■ Trees

- A tree (or free tree) is a connected acyclic graph.
- Forests: a graph that has no cycles but is not necessarily connected.

■ Properties of trees

- $|E| = |V| - 1$
- For every two vertices in a tree there always exists exactly one simple path from one of these vertices to the other.
 - **Rooted trees:** The above property makes it possible to select an arbitrary vertex in a free tree and consider it as the root of the so-called rooted tree.
 - Levels of rooted tree.

Rooted Trees

- ancestors
 - For any vertex v in a tree T , all the vertices on the simple path from the root to that vertex are called ancestors.
- descendants
 - All the vertices for which a vertex v is an ancestor are said to be descendants of v .
- parent, child and siblings
 - If (u, v) is the last edge of the simple path from the root to vertex v (and $u \neq v$), u is said to be the parent of v and v is called a child of u .
 - Vertices that have the same parent are called siblings.
- Leaves
 - A vertex without children is called a leaf.
- Subtree
 - A vertex v with all its descendants is called the subtree of T rooted at v .

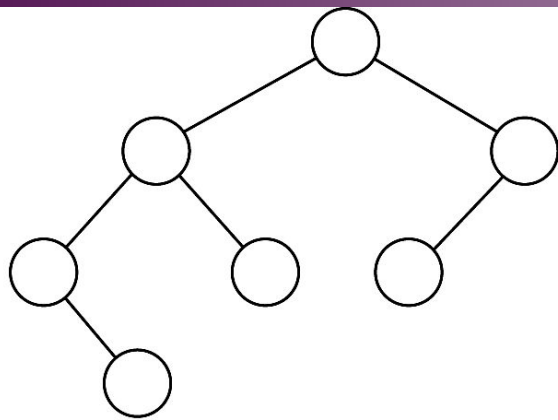


Trees

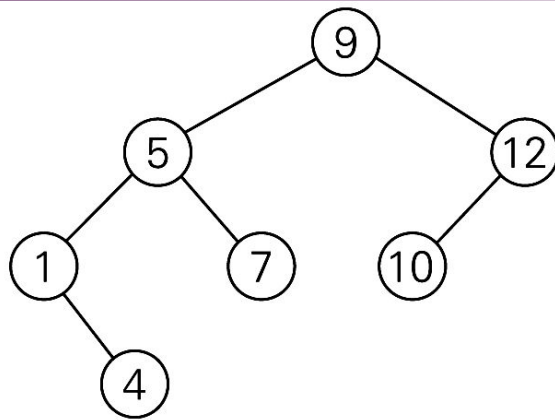
- Depth of a vertex
 - The length of the simple path from the root to the vertex.
- Height of a tree
 - The length of the longest simple path from the root to a leaf.

Ordered Trees

- Ordered trees
 - An ordered tree is a rooted tree in which all the children of each vertex are ordered.
- Binary trees
 - A binary tree is an ordered tree in which every vertex has no more than two children and each children is designated as either a left child or a right child of its parent.
- Binary search trees
 - Each vertex is assigned a number.
 - A number assigned to each parental vertex is larger than all the numbers in its left subtree and smaller than all the numbers in its right subtree.
- $\lceil \log_2 n \rceil \leq h \leq n - 1$, where h is the height of a binary tree.

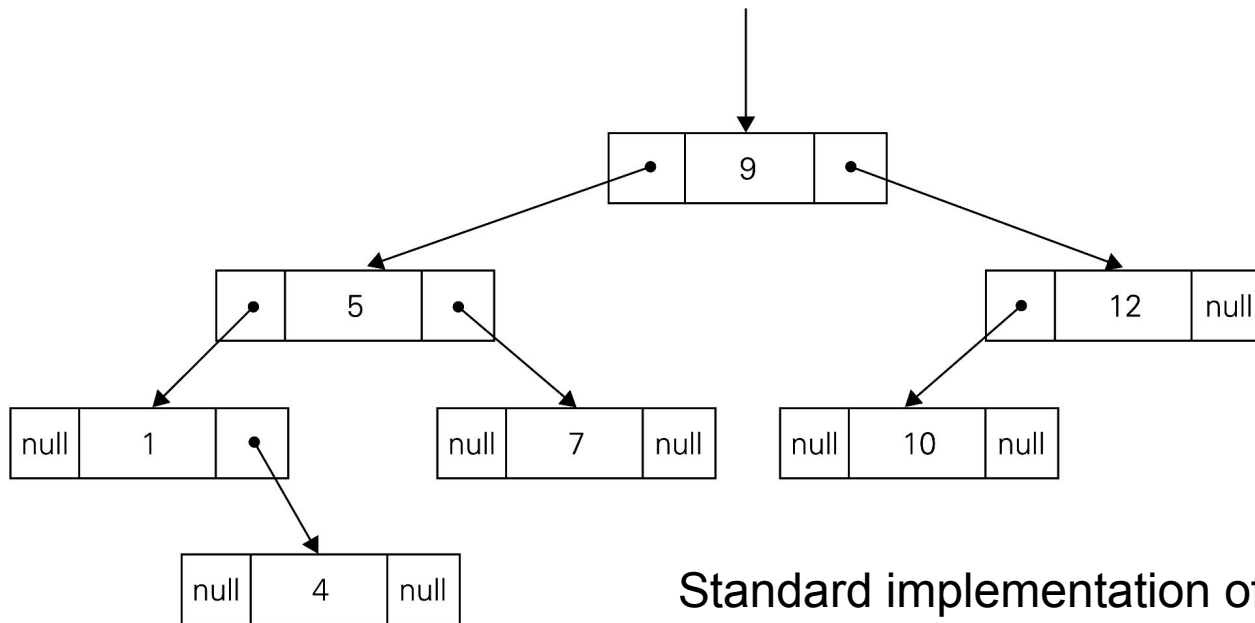


(a)

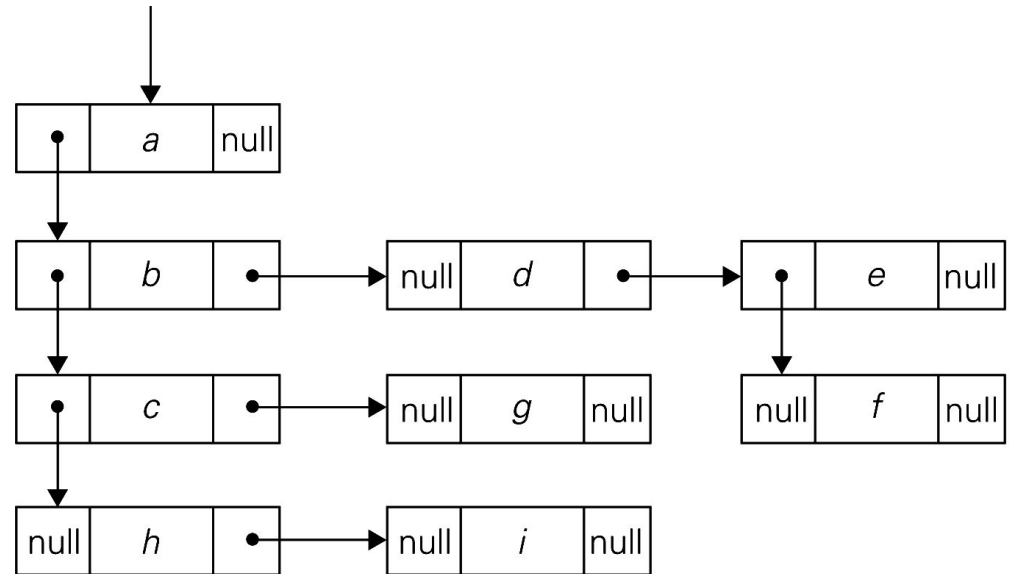
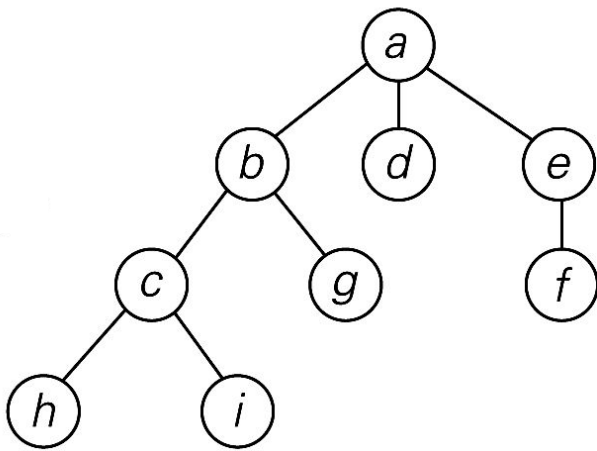


(b)

A binary tree and a binary search tree



Standard implementation of (b)



A rooted tree and its first child-next sibling representation

Fundamental data structures

Sets, Bags, Dictionaries

- **Set:** unordered collection of distinct elements
Operations: membership, union, intersection
Representation: bit string; linear structure
- **Bag:** unordered collection, elements may be repeated
- **Dictionary:** a bag with operations search, add, delete

a **bag** focuses on counting occurrences of elements, while a **dictionary** associates keys with specific values.



Conclusion

- **Algorithm classification**

- By types of problems
- By design technique

- **Design techniques**

- a general approach to solving problems